LINUX TERMINAL CHAT

嵌入式 Linux 终端聊天室

C 语言 / TCP Socket / select / 多客户端消息转发

项目目标是在 Linux 终端中实现一个小型聊天室。一个进程负责接收与转发消息，多个客户端进程负责文字输入和显示。同一份 C 源码通过启动参数选择服务器或客户端角色。

完成情况

已实现：最多 16 个客户端群聊、自动用户编号、加入与退出提示、中文 UTF-8 消息传输、/quit 退出、按换行划分消息、异常输入处理。

15 / 15

自动集成测试全部通过

GCC 严格编译通过，无编译警告

包含真实 C 客户端的发送、接收和退出

验证范围

测试在 Windows 电脑上的 QEMU Linux 虚拟机内完成，系统为 Alpine Linux 3.24.3，架构 x86_64。服务器和客户端均为实际编译运行的 C 程序。Python 仅用于测试驱动与结果记录。

尚未验证：鲁班猫实机运行、两台实体设备之间的通信和长时间压力运行。本文不会把虚拟机测试写成开发板实测。

作业边界

本项目按终端聊天作业准备。教师的详细功能清单、是否必须上板、是否必须提交 GitHub，尚未获得确认。本文不含姓名、学号或班级信息。

公开源码仓库

https://github.com/starsstring/linux-terminal-chat

扫描右侧二维码，或点击上方链接查看源码与报告。

01 程序结构与通信流程

一个服务器 + 多个客户端；本机演示不依赖开发板



消息路径示例：客户端 A 输入文字，服务器读取完整的一行，再将带用户编号的消息发给所有在线客户端。发送者也收到自己的消息回显。连接支持双向通信。

服务器的工作循环

socket 创建通信端点；bind 绑定端口；listen 等待连接。select 监听服务器套接字和已有客户端连接：有新连接时 accept；有消息时 recv；形成完整消息后 broadcast。

客户端的工作循环

connect 连接服务器。select 同时等待标准输入和网络连接：键盘可读时 read 并发送，网络可读时 recv 并显示。无需为每个输入来源创建线程。

连接状态

每个客户端保存连接描述符 fd、用户编号 id、缓冲区使用长度 used 和当前消息缓冲区 line。连接关闭时释放槽位；新加入的用户获得新的编号。

本机演示与跨设备演示

127.0.0.1 指向当前 Linux 系统，可以在一台电脑的 Linux 中开三个终端，运行一个服务器和两个客户端。跨设备时，客户端应使用服务器实际的局域网 IPv4 地址；这需要额外验证网络和防火墙。

02 消息协议与关键处理

消息按换行分隔；TCP 的读取边界不等于聊天消息边界

应用层协议

客户端发送：消息正文 + \n
服务器广播：[User 编号] 消息正文 + \n
退出命令：/quit + \n

消息正文最多 1023 字节。中文以 UTF-8 字节传输，因此字节数不能直接当作汉字数量。CRLF 输入中的回车会被忽略，换行结束一条消息。

TCP 分段和合并

一次发送可能分多次到达，多次发送也可能一起到达。服务器把收到的字节逐个放入对应用户的缓冲区，只有遇到换行才转发；一次读取出现多行时逐行处理。

分段到达："frag" + "ment\n" → 一条 fragment 消息
合并到达："one\ntwo\n" → 两条独立消息

发送不完整与断开

send_all 循环发送尚未发送的字节，处理部分发送和 EINTR。程序忽略 SIGPIPE，让发送失败可以走错误处理。recv 返回 0 表示对方关闭连接；服务器释放该用户，客户端则结束运行。

边界与异常

端口必须在 1 到 65535 范围内；客户端要求 IPv4 地址。超长消息或无效控制字符会断开该客户端；容量超过 16 人时拒绝新连接。发送设置两秒超时，但慢客户端仍可能暂时拖慢整个服务器。

03 编译、运行与演示

在电脑 Linux 中运行；以后可复制源码到鲁班猫重新编译

编译

```
gcc -std=c11 -Wall -Wextra -Wpedantic -O2 chat.c -o chat
```

在源码文件夹中执行。具备 make 时也可以运行 make。测试需要 Python 3；聊天程序本身不依赖 Python 。Windows 普通程序不能直接当作鲁班猫程序运行。

三个终端启动

```
终端一：./chat server 8888
终端二：./chat client 127.0.0.1 8888
终端三：./chat client 127.0.0.1 8888
```

先启动服务器，再启动两个客户端。分别输入 Hello 和中文消息，观察两个客户端都收到消息。输入 /quit 退出客户端；按 Ctrl+C 停止服务器。

自动演示和回归测试

```
python3 demo_chat.py
make test
```

演示脚本启动真正的 C 服务器和两个 C 客户端，自动输入消息并保存输出；测试脚本检查基本行为和边界条件。test-results.json 记录环境、测试条目与源码 SHA-256。

不需要开发板的复现方式

本次使用独立 QEMU Linux 环境进行验证。附带的 Windows 演示入口会使用同目录中的 Linux 演示镜像启动三个终端分区。它运行的是 Linux 程序，不是 Windows 版聊天程序。具体按演示说明操作。

后续上板时先检查 uname -m 与 command -v gcc，再复制源码、编译并重复相同验收步骤。迁移方案不等于上板结果。

04 自动测试结果

严格编译通过；下表为实际执行结果，非计划清单

环境：Alpine Linux 3.24.3 / Linux-6.18.58-0-virt-x86_64-with-musl1
编译器：gcc (Alpine 15.2.0) 15.2.0

测试项目	结果
01 自动用户编号	PASS
02 双向群聊与消息回显	PASS
03 UTF-8 中文字节收发	PASS
04 TCP 分段到达	PASS
05 TCP 多消息合并到达	PASS
06 CRLF 换行兼容	PASS
07 1023 字节消息边界	PASS
08 超长消息断开且其他用户可用	PASS
09 /quit 退出和重新加入	PASS
10 拒绝终端控制字符	PASS
11 16 客户端和容量上限	PASS
12 非法参数和错误 IPv4	PASS
13 真实 C 客户端收发及退出	PASS
14 客户端标准输入结束	PASS
15 服务器停止后客户端退出	PASS

统计：15 / 15 项通过。编译增加 -Werror，把警告作为错误处理；未发现编译警告。

对应源码 SHA-256：
ba57840d10ffbc82f482f843a890d9b551bbedebbaaef722fee95c49e7695cba

这些结果不覆盖鲁班猫实机、跨设备网络、字体渲染、长期稳定性或恶意网络攻击。GitHub Actions 工作流已准备；发布前不能声称云端测试已经执行。

05 实际运行输出

以下为自动演示中真实 C 进程的输出节选，经排版呈现

测试驱动向客户端 A 输入 Hello from terminal A!，向客户端 B 输入中文问候，再向客户端 A 输入 /quit。以下文字直接取自 demo-output.json；不是手写的预期结果。

客户端 A 的标准输出

```
[system] You are User 1. Enter text; /quit to leave.
[system] User 1 joined.
[system] User 2 joined.
[User 1] Hello from terminal A!
[User 2] 你好，终端 A！
[system] Connection closed.
```

客户端 B 的标准输出

```
[system] You are User 2. Enter text; /quit to leave.
[system] User 2 joined.
[User 1] Hello from terminal A!
[User 2] 你好，终端 A！
[system] User 1 left.
```

观察结果

两个客户端均收到带用户编号的英文和中文消息。客户端 A 退出后，客户端 B 收到 User 1 left. 通知。演示驱动对双向消息和离开提示进行了断言。

原始标准输出和输入事件分别保存在 demo-output.txt 与 demo-output.json 中，可以重新执行 demo_chat.py 复现。终端字体与窗口输入体验仍应在实际演示电脑上检查。

06 项目总结与交付说明

已完成的软件功能、尚未验证的范围与答辩要点

项目价值与限制

本作业把 C 语言、进程、标准输入输出、TCP Socket 与 select 结合到一个可演示的小程序中。核心程序只有一份源文件，方便跟随 main、server、client、send_all 和 broadcast 阅读。

本版不包含账号认证、私聊、聊天记录、文件传输或加密。普通终端收到消息时可能打断正在输入的文字显示。服务端为教学规模设计，不作为公网服务使用。

答辩时应能讲清楚

为什么选 TCP：有序的字节传输适合文字收发，但应用仍需处理消息边界。
为什么用 select：一个循环等待多个连接，以及客户端的键盘和网络输入。
为什么用换行：TCP 不保留消息边界，需要自己约定分隔符。
能否说已经上板：不能；目前是电脑 Linux 虚拟环境的实测。

交付文件

chat.c 与 Makefile：源代码和构建入口。
test_chat.py 与 test-results.json：测试方法和实测结果。
demo_chat.py 与 demo-output 文件：自动演示和输出记录。
README、代码讲解、演示步骤：复现、阅读和讲解说明。
本报告与项目压缩包：用于整理提交材料。

GitHub 与课程入口

公开仓库：https://github.com/starsstring/linux-terminal-chat
源码、测试脚本、实测记录与本报告一同提交到仓库，报告位于 docs/report.pdf。首页附可点击链接与二维码；课程入口只接收 PDF 时，可使用本报告。教师是否要求 GitHub 或实机演示，以课程通知为准。

环境参考

QEMU 官方下载说明：https://www.qemu.org/download/
Alpine Linux 官方镜像目录：https://dl-cdn.alpinelinux.org/alpine/
测试依据：随项目提供的源码哈希、test-results.json 和实际演示日志。